TECHNICAL HANDBOOK / 2026

TraceMind

From application logs to evidence-backed investigation

Architecture, deterministic correlation, RCA contracts,
API reference, configuration, migrations and operations.

Expanded from the project README and verified source code.

Prepared 20 October 2026 | Repository revision 6c33e25

Documentation snapshot - runtime state must be checked before deployment.

Contents

01 Overview 3

02 Architecture 4

03 Key Features 7

04 Tech Stack 13

05 Prerequisites 14

06 Getting Started 15

07 Environment Variables 18

08 API Endpoints 19

09 Database Migrations 21

10 Testing 23

11 Project Structure 25

12 Future Scope 26

13 License 27

14 Worked Investigation 27

15 Troubleshooting & Source Notes 28

How to use this document

The thirteen main chapters follow the README's organization. Additional explanations are placed at the end of each chapter. The final walkthrough and source notes provide a worked investigation example, troubleshooting guidance, and the scope of this documentation snapshot.

Provider-free commands and live analysis are distinguished. All live provider examples are instructions for a future authorized run; no external analysis was performed to create this handbook.

01 Overview

TraceMind is an incident-investigation backend that ingests structured application logs, groups related failures into incidents, and generates evidence-backed root-cause hypotheses. It combines deterministic correlation with optional LLM analysis while preserving original evidence and clearly labeling causality as unconfirmed.

The current implementation includes validated log ingestion, deterministic incident detection, strictly validated Gemini or OpenAI RCA reports, immutable evidence snapshots, structured diagnostics, reproducible dependencies and container images, and explicit Alembic migration management.

Why TraceMind

A log viewer can show that several services failed near the same time, but it does not explain whether the failures belong to the same incident or which evidence supports a possible cause. TraceMind separates that work into two controlled stages:

1. Deterministic code groups and relates logs using identifiers, time boundaries, fingerprints, and configured service dependencies.
2. An optional LLM explains a bounded evidence package and returns a structured hypothesis whose citations and semantics are validated before storage.

The LLM does not decide which raw logs belong to an incident, modify incidents, confirm root causes, or execute remediation.

Reading this handbook

This guide combines the repository README with implementation-based explanations for developers and operators. The original product, API, environment, migration and test details are retained. Rendered diagrams and relational tables replace Mermaid source so the PDF is usable independently of GitHub.

Read Architecture and Key Features to understand the data flow. Use Getting Started and Environment Variables for local setup, Database Migrations for controlled adoption, and Testing for repeatable checks. The later operational walkthrough explains how these pieces work together.

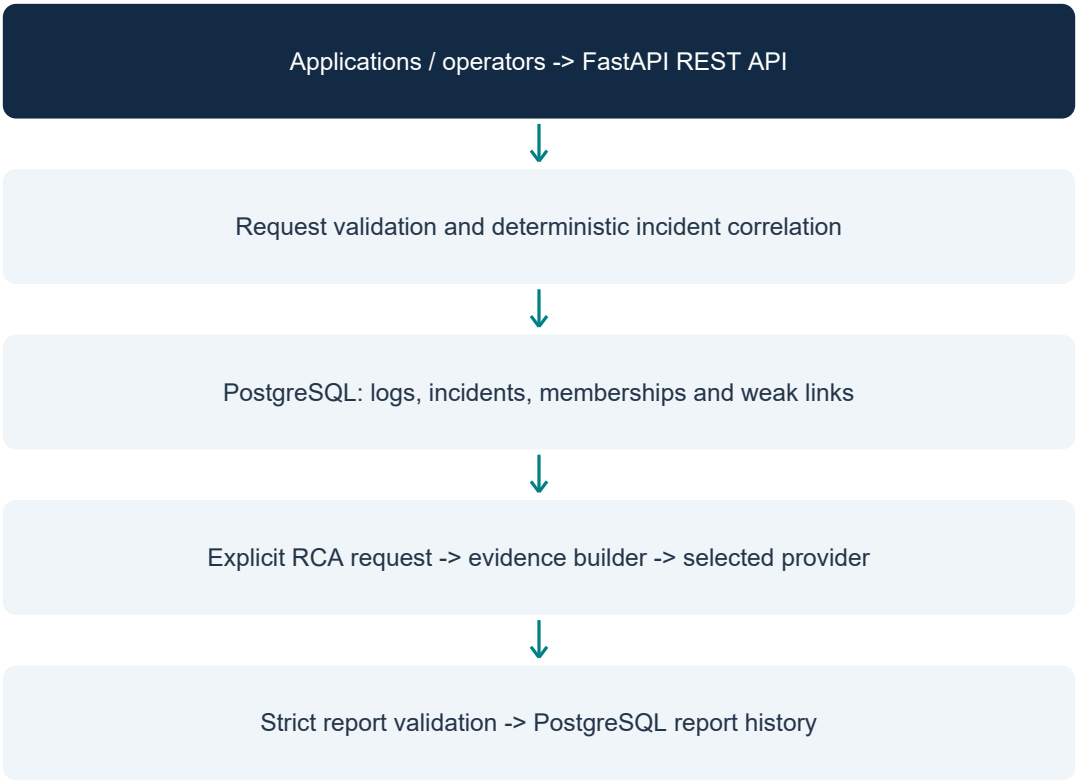
What evidence-backed means

Each RCA report is tied to a particular snapshot of supplied logs. A citation can be checked mechanically against that snapshot. This proves traceability, but it does not prove that the log supports the provider's interpretation. An operator still needs to inspect the cited events, challenge alternatives, and carry out recommended checks before accepting a cause.

The application therefore separates observations, hypotheses and workflow status. A resolved incident is a lifecycle state; it is not a verified causal conclusion. Confidence labels are qualitative judgments, not percentages or calibrated probabilities.

02 Architecture

System context



PostgreSQL is the system of record; provider output must pass validation.

PostgreSQL is the system of record. Provider calls are synchronous and occur only when a client explicitly requests analysis. `POST /logs` and `POST /incidents/detect` never invoke an LLM.

Internal component architecture

Layer	Modules and role
HTTP boundary	main.py, incidents.py, rca_routes.py: startup, health and REST routing
Domain processing	correlation.py and rca.py: grouping, evidence, validation and persistence
Provider boundary	llm.py: selected Gemini/OpenAI transport and response handling
Contracts	schemas.py and rca_schemas.py: request, response and report structure
Persistence/configuration	database.py, models.py, config.py: sessions, tables and settings
Maintenance	migrations.py plus Alembic scripts: explicit upgrade and startup revision checks

Component responsibilities

Component	Responsibility
FastAPI routes	Validate HTTP input, invoke domain services, serialize responses, and map failures to sanitized status codes
Pydantic schemas	Enforce request and response types, bounds, enums, unknown-field rejection, and strict RCA output structure
Correlation engine	Assign unprocessed logs to incidents, calculate fingerprints, apply identifier and time rules, emit signals, and store weak relationships
RCA service	Select bounded evidence, call the provider, parse output, enforce semantic and citation rules, and save immutable reports
Provider adapters	Translate the common RCA contract to Gemini or OpenAI request formats and safely handle provider failures
SQLAlchemy models	Represent logs, incidents, memberships, incident links, and saved RCA reports
PostgreSQL	Persist source evidence and investigation history across API restarts

Data model

Table	Key and relationships	Stored information
logs	Integer primary key; referenced by memberships and links	Timestamp, service, severity, message, request/trace IDs, exception type
incidents	Integer primary key; parent of memberships and reports	Title, severity, lifecycle status, time range, occurrence count, rules, timestamps
incident_logs	log_id primary key; incident_id and related_log_id foreign keys	One membership per log, reason, strength and related evidence
incident_links	Composite upstream/downstream log primary key; references both incidents	Weak relationship, direction, reason and strength
rca_reports	Integer primary key; incident_id foreign key	Provider/model, prompt version, analysis, snapshot/hash, notes, duration, creation time

Logical relationships: incidents have many memberships and reports. Each log has at most one membership; a membership may reference a related log. Links point from an upstream log/incident pair to a downstream pair. SQLAlchemy uses integer IDs; the README diagram is a conceptual view.

`incident_logs` ensures that each log belongs to at most one incident. Incident links record weaker cross-incident evidence without merging those incidents. Each RCA report stores the exact evidence snapshot and its SHA-256 hash, so later log ingestion or status changes do not rewrite previous analysis history.

End-to-end processing flow



Provider invocation occurs only on an explicit analysis request with usable evidence.

If provider output fails any validation step, no report row is created.

Storage and transaction boundaries

Log ingestion commits a complete validated batch. Detection persists grouping decisions independently of RCA. Report generation reads evidence, obtains a candidate, validates it and commits the report only after acceptance. These separate boundaries allow logs and incidents to remain useful even when RCA is disabled or a provider fails.

The evidence hash identifies the serialized evidence snapshot, not the truth of a hypothesis. Persisting that snapshot alongside analysis prevents later log changes from silently changing what a historical report was based on. This is application-level history preservation; it is not cryptographic protection against a database administrator editing records.

Three different operational checks

Check	Question answered
Startup revision verification	Does the recorded database revision exactly match the heads in the packaged migration scripts?
HTTP health probe	Can the running API currently query the database?
Schema comparison	Does the physical schema match ORM metadata? This is a separate maintenance/audit check.

The startup guard reads migration history; it does not run a full schema-drift inspection. A manually stamped but incorrect schema can pass a revision check. That is why schema comparison is required before adopting migrations on an existing database.

03 Key Features

Log ingestion

Log contract

`POST /logs` accepts an object containing a `logs` array with 1-1,000 records. Every record requires `timestamp`, `service`, `severity`, and `message`. Optional fields are `request_id`, `trace_id`, and `exception_type`. Unknown fields are rejected. Validation is atomic: one invalid entry rejects the complete batch with HTTP 422.

Normalization rules:

- Timestamps must contain timezone information and are stored and returned in UTC.
- Service names are trimmed, lowercased, and normalized from spaces or underscores to hyphens.
- Services may contain letters, digits, dots, and hyphens and are limited to 100 characters.
- Severity is case insensitive. `WARN` becomes `WARNING`; `FATAL` becomes `CRITICAL`.
- Messages are trimmed, required to be nonempty, and limited to 32,000 characters.
- Optional identifiers must be nonempty when supplied and are limited to 200 characters.

Ingestion does not deduplicate records. Repeating the same request stores another set of observations.

Ingest and query sample logs

```
$sampleBody = Get-Content -Raw data/sample_logs.json
Invoke-RestMethod -Method Post -Uri http://localhost:8000/logs `
  -ContentType 'application/json' -Body $sampleBody

Invoke-RestMethod 'http://localhost:8000/logs?service=payment-service&severity=ERROR&limit=20&offset=0'
Invoke-RestMethod 'http://localhost:8000/logs?start=2026-10-17T10:00:01Z&end=2026-10-17T10:00:03Z'
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

`GET /logs` supports combinable `service`, `severity`, inclusive `start`, inclusive `end`, `limit`, and `offset` parameters. Results sort by timestamp and then ID in ascending order. Encode `+` as `%2B` when a timestamp with a positive offset appears in a URL.

Incident detection and correlation

Detection is explicit. Call it after ingestion:

```
$detection = Invoke-RestMethod -Method Post http://localhost:8000/incidents/detect
$incidents = Invoke-RestMethod http://localhost:8000/incidents
$incidents.items | Format-Table id, title, severity, status, occurrence_count
$incidentId = $incidents.items[0].id
Invoke-RestMethod "http://localhost:8000/incidents/$incidentId"
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The detector processes unassigned logs and returns counts for created incidents, assigned logs, and weak relationships. Existing Phase 1 logs can be detected without being submitted again.

Correlation precedence and safeguards

1. Exact `trace_id` matches provide the strongest grouping rule.
2. Exact `request_id` matches are used when trace evidence does not conflict.
3. Different nonempty trace IDs prevent grouping through a shared request ID.
4. An incident has a fixed maximum span controlled by `CORRELATION_WINDOW_SECONDS`; chains of nearby events cannot extend it indefinitely.
5. Identifier-free errors can group only when service, exception type, and normalized message fingerprint match within the window.
6. Fingerprinting normalizes changing numeric values, UUIDs, and hexadecimal addresses so repeated forms of one error can group.
7. ERROR and CRITICAL logs seed incidents. Lower-severity records attach only as identifier-matched context.
8. Repeated fingerprints, multi-service traces, and error bursts add transparent detection signals.
9. A timeout or latency event in a downstream service may link to an earlier dependency failure as weak evidence. It does not merge the incidents or prove causality.
10. Conflicting request or trace identifiers block weak dependency links.

The sample dataset groups the database and payment errors sharing `trace-payment-001`. Its unrelated user-service warning remains queryable but does not seed that incident.

Detection is idempotent for stored log IDs. It does not reassign or retroactively merge existing incidents. New failures do not attach to resolved incidents; recurrence creates another incident.

Incident lifecycle

Statuses are `open`, `investigating`, and `resolved`.

```
Invoke-RestMethod 'http://localhost:8000/incidents?service=payment-service&status=open&limit=10'

Invoke-RestMethod -Method Patch -Uri "http://localhost:8000/incidents/$incidentId" `
  -ContentType 'application/json' -Body '{"status":"investigating"}'

Invoke-RestMethod -Method Patch -Uri "http://localhost:8000/incidents/$incidentId" `
  -ContentType 'application/json' -Body '{"status":"resolved"}'
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

A status change records workflow state only. It does not confirm a cause or save a verified resolution.

Evidence-backed AI-assisted RCA

Phase 3 builds a bounded evidence package from an existing incident and asks the selected provider for a strict JSON report. The backend then validates that report before saving it.

RCA pipeline

1. Load the incident, its original logs, membership reasons, detection rules, and eligible weak relationships.

2. Select ERROR/CRITICAL logs first, followed by lower-severity context, while applying log-count and character limits.
3. Record evidence omissions and truncation as explicit evidence gaps.
4. Send the evidence and system instructions to the selected provider.
5. Decode and validate the response with `RCAOutput`.
6. Verify every cited log ID belongs to the supplied evidence snapshot and reject duplicate citation IDs.
7. Reject invented services, overlapping supporting/contradicting citations, invalid ranks, unsupported statuses, and unknown fields.
8. For a hypothesis, require at least one observed fact, reject duplicate cause text case-insensitively, and require confidence to be non-increasing by rank.
9. Require `probable_cause` to equal the first ranked cause and require its top-level citation list to match the first cause's citations exactly, including order.
10. Convert a report with contradictory evidence on its first-ranked cause to inconclusive, clear the selected cause and top-level citations, and lower confidence.
11. Merge snapshot omission and truncation gaps into the final report, remove duplicates while preserving order, and keep at most 20 gaps.
12. Persist the analysis, evidence snapshot, hash, provider, model, prompt version, validation notes, and total duration in one report transaction.

Reports can have `hypothesis` or `inconclusive` status. They cannot claim `confirmed`. Every API response retains `causality: "unconfirmed"`.

Gemini configuration

```
RCA_PROVIDER=gemini
GEMINI_API_KEY=your-local-key
GEMINI_MODEL=gemini-3.5-flash-lite
RCA_TIMEOUT_SECONDS=30
RCA_MAX_LOGS=40
RCA_MAX_EVIDENCE_CHARS=40000
RCA_MAX_OUTPUT_TOKENS=4000
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The adapter uses Google's official `v1beta/models/{model}:generateContent` endpoint through HTTPX. It sends the key in the `x-goog-api-key` header and uses native `systemInstruction`, `contents`, and `generationConfig` fields. JSON output is requested with `responseMimeType: application/json` and a translated `responseJsonSchema`.

The generated schema retains closed objects, required fields, enums, nullability, minimum array sizes, root array limits, and numeric rank bounds. Controlled live testing found that Google's schema validator rejected the complete RCA schema when three nested citation arrays included `maxItems`. Those three generation-time bounds are omitted, while the unchanged Pydantic model still enforces their 100-ID limits before persistence. Details and probe results are recorded in docs/gemini-investigation.md.

Only one candidate ending with `STOP` is accepted. Blocked, incomplete, empty, oversized, malformed, or unexpected responses are rejected. There are no automatic retries, provider fallbacks, model switches, tools, or search calls.

Gemini availability, free-tier eligibility, and limits depend on the Google project. Consult Gemini pricing, rate limits, and structured output documentation.

OpenAI configuration

```
RCA_PROVIDER=openai
OPENAI_API_KEY=your-local-key
OPENAI_MODEL=your-structured-output-capable-model
RCA_TIMEOUT_SECONDS=30
RCA_MAX_LOGS=40
RCA_MAX_EVIDENCE_CHARS=40000
RCA_MAX_OUTPUT_TOKENS=4000
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The OpenAI adapter uses the Responses API with strict structured output and `store: false`. Only the selected provider requires credentials. Set `RCA_PROVIDER=disabled` to stop new analyses while keeping saved reports readable.

Generate and retrieve an RCA report

```
$incidentId = 1
$report = Invoke-RestMethod -Method Post `
  -Uri "http://localhost:8000/incidents/$incidentId/analyze" `
  -ContentType 'application/json' -Body '{"max_logs":40}'

$report.analysis | ConvertTo-Json -Depth 10
Invoke-RestMethod "http://localhost:8000/incidents/$incidentId/analyses?limit=20&offset=0"
Invoke-RestMethod "http://localhost:8000/incidents/$incidentId/analyses/$($report.id)"
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The request body is optional. `max_logs` must be between 1 and 100 and cannot exceed the server limit. Every successful request creates a new immutable report and makes a provider call when usable evidence exists. Retrieval never calls the provider.

New reports use prompt version `rca-v2`, which explicitly requires the probable cause to be a verbatim copy of the first ranked cause. Existing reports retain the prompt version stored at creation.

Trust and failure boundaries

- Logs, identifiers, exception text, and incident titles are treated as untrusted data, not instructions.
- Provider credentials are never put in URLs, response bodies, or application diagnostics.
- Public failures do not expose provider bodies, authorization headers, incident evidence, or API keys.
- Gemini diagnostics log only HTTP status, an allowlisted provider status, and a bounded classified description.
- OpenAI diagnostics use fixed failure categories and omit response bodies and exception details.
- RCA orchestration records provider-call elapsed time with a monotonic clock for successful and failed calls. Validation and persistence failures use stable categories.
- Diagnostic records include bounded provider, model, prompt-version, status, and timing fields where applicable. They exclude full prompts, raw evidence, raw provider output, authorization headers, API keys, and unsanitized response bodies.
- Provider response envelopes and extracted report text have hard size limits.
- Report metadata is created only after provider output passes validation. A report is returned and logged as persisted only after the database commit succeeds; persistence errors roll back and retain the controlled API error.
- Failed calls, rejected output, and failed commits do not leave report rows.
- A cited log proves only that the log was supplied and referenced; human review is still required to judge whether it supports the claim.

Provider requests send selected incident evidence to the configured external service. Review log contents and the provider's data terms before enabling RCA.

Reliability and deterministic validation

Phase 4's deterministic RCA validation and reliability audit is complete. The implemented boundary includes ordered citation consistency, observed-fact requirements for hypotheses, duplicate-cause rejection, non-increasing confidence by rank, contradictory-evidence handling, normalized evidence gaps, persistence rollback coverage, immutable report metadata, sanitized structured diagnostics, provider no-retry tests, and the fixture-backed `rca_benchmark` suite.

This phase establishes repeatable contract validation and failure handling. It does not establish that a provider's proposed cause is correct, compare model quality, benchmark request throughput or latency, exercise live provider services, add reviewed causes or resolutions, or make RCA processing asynchronous. Those remain separate product and evaluation concerns.

Detailed validation examples

Candidate or evidence situation	Deterministic outcome
Top-level citations [12, 13], first cause citations [13, 12]	Reject: IDs are identical but their order differs.
Citation refers to log 99, which is absent from the snapshot	Reject even if log 99 exists elsewhere in the database.
Hypothesis has an empty observed_facts array	Reject; a hypothesis must include at least one fact with citations.
Causes differ only by letter case	Reject duplicate causes after schema trimming and casefold comparison. This is not semantic paraphrase detection.
Rank 1 is low confidence and rank 2 is medium	Reject increasing confidence before normalization.
First-ranked cause has contradicting citations	Convert to inconclusive; clear selected cause and top-level citations.
Sparse snapshot or snapshot omission gaps	Limit report and cause confidence to low.
Provider returns high confidence	Cap it at medium because the available evidence consists of logs.

The implementation does not require top-level confidence to equal the first cause's confidence. It applies the documented caps and evidence-based normalization to both. Adding equality would invent a relationship not enforced by the current contract.

RCA report contract in detail

All report fields are required in provider output and unknown fields are rejected. Text fields are stripped and bounded to 4,000 characters. Hypothesis and inconclusive are the only allowed statuses. Confidence is one of low, medium or high.

Field group	Bound and meaning
observed_facts	Up to 30 observations; each has 1-100 supporting integer log IDs. Hypotheses require at least one.

Field group	Bound and meaning
ranked_causes	Up to 5 causes, contiguous ranks beginning at 1, explanation, confidence, supporting and contradicting IDs.
supporting_log_ids	Top-level citations, at most 100. A hypothesis must exactly match its first cause's list.
affected_services	Up to 100 service strings, checked against the evidence and then normalized to snapshot services.
alternative_explanations	Up to 10 entries. These are possibilities for human review.
recommended_checks	Between 1 and 15 next checks; the application does not execute them.
evidence_gaps	Up to 20 entries. Snapshot gaps precede report gaps; duplicates are removed while preserving order.

Supporting and contradicting lists for the same cause cannot overlap. Every citation list is checked for duplicated IDs. An inconclusive output must not assert a probable cause or top-level supporting citations; if its gaps are empty, the validator supplies an explanation.

No usable evidence

When analysis reaches report creation without usable logs, the application builds a local inconclusive result with low confidence and an evidence gap explaining the absence. It does not call the provider. Provider selection and request configuration still apply at the route boundary; this is not a separate mock-provider mode available to API clients.

04 Tech Stack

Area	Technology
API	Python 3.11+, FastAPI, Uvicorn
Validation and settings	Pydantic, pydantic-settings
Persistence	PostgreSQL 17, SQLAlchemy 2, psycopg 3
Provider HTTP	HTTPX
AI providers	Google Gemini `generateContent`, OpenAI Responses API
Dependency management	uv 0.13.1 with a universal lock
Testing and quality	pytest, Ruff
Runtime packaging	Docker, Docker Compose

No Kafka, Kubernetes, Redis, vector database, provider SDK, or frontend is required by the current implementation.

Reproducibility responsibilities

pyproject.toml remains the source for direct dependency requirements. uv.lock records a particular complete resolution, hashes and platform-specific distributions. The dev extra adds pytest and Ruff; the runtime image omits that extra. The tool itself is constrained to uv 0.13.1.

The project permits Python 3.11 and later in metadata, while the documented verification workflow uses Python 3.13. A universal lock supports platform-dependent resolution; it does not mean every Python version and operating system combination has been executed in CI.

Python 3.13.16, uv 0.13.1 and PostgreSQL 17.11 are pinned by version and manifest digest. Digests provide repeatable image selection, but fixes and security updates still require deliberate review and a new validated digest. Build commands do not erase the PostgreSQL volume.

05 Prerequisites

- Docker Desktop with Docker Compose for the container workflow.
- PowerShell for the documented Windows commands.
- Python 3.13 and uv 0.13.1 for local development outside Docker.
- PostgreSQL 17 for local database-backed development and testing.
- A Gemini or OpenAI API key only when explicitly enabling that provider. Log ingestion, correlation, stored-report retrieval, and the automated tests do not require provider credentials.

Choose a workflow

Use the Docker workflow when you want the packaged Python runtime and dependency set. Use local Python when editing code and running tests on Windows. Both workflows can use the Compose PostgreSQL service; the application running on the host connects through localhost, while an API container connects to the service name db.

Ports 8000 and 5432 must be available for the default Compose configuration. Commands in this document assume the repository root is the current directory. Settings are loaded from the process environment and the local .env file; the API container receives the Compose environment overrides.

06 Getting Started

Docker

```
docker compose build api
docker compose up -d --wait db
docker compose run --rm api python -m app.migrations upgrade
docker compose up -d --wait api
Invoke-RestMethod http://localhost:8000/health
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

These commands assume a new empty database. For an existing unversioned database, complete the baseline procedure below before starting the updated API.

Open the interactive API documentation at http://localhost:8000/docs.

The Compose stack exposes the API and PostgreSQL only on localhost. Database data persists in the Compose volume `postgres_data`, named `tracemind_postgres_data` when the default project name is used. The included database credentials are for local development. The API has no authentication and should not be exposed publicly.

Useful lifecycle commands:

```
docker compose ps
docker compose logs --tail 100 api
docker compose restart api
docker compose down
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Both services use `restart: unless-stopped`. Docker restarts them after a process, daemon, or host restart, while an explicit `docker compose stop` keeps them stopped. `docker compose down` still removes the containers and network normally while preserving the named database volume.

`docker compose ps` reports `(healthy)` for both `db` and `api` when PostgreSQL accepts connections and the API's internal `http://127.0.0.1:8000/health` probe returns HTTP 200. The API probe uses the Python standard library included in the image; HTTP 503 and connection failures make the container unhealthy. Inspect the latest probe output when troubleshooting:

```
docker inspect --format='{{json .State.Health}}' tracemind-api-1
docker compose logs --tail 100 api db
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

`docker compose down` preserves the named database volume. Do not add `--volumes` when you want to retain data.

To verify persistence with a harmless log record, save its returned ID and retrieve it after both a restart and a container recreation:

```
$marker = "phase5-persistence-$([DateTimeOffset]::UtcNow.ToUnixTimeMilliseconds())"
$body = @{ logs = @(@{
  timestamp = [DateTime]::UtcNow.ToString('o')
  service = $marker
  severity = 'INFO'
  message = 'Phase 5 persistence verification'
}) } | ConvertTo-Json -Depth 4
$record = (Invoke-RestMethod -Method Post -Uri http://localhost:8000/logs `
  -ContentType 'application/json' -Body $body)[0]

docker compose restart db
docker compose up -d --wait
(Invoke-RestMethod "http://localhost:8000/logs?service=$marker").items.id

docker compose up -d --force-recreate db
docker compose up -d --wait
(Invoke-RestMethod "http://localhost:8000/logs?service=$marker").items.id
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Both retrieved IDs should equal `$record.id`. These commands recreate the PostgreSQL container but do not remove `tracemind_postgres_data`. Never add `--volumes` to this verification workflow.

Local development

Python 3.13 and PostgreSQL are used for the verified development workflow. Install the project-required uv version explicitly, then synchronize runtime and development dependencies from `uv.lock`:

```
py -3.13 -m pip install --user uv==0.13.1
uv --version
uv lock --check
uv sync --frozen --extra dev --python 3.13
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
docker compose up -d db
uv run --frozen uvicorn app.main:app --reload
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

`--frozen` makes `uv.lock` the installation source of truth and prevents an install or command from silently changing dependency resolution. `uv lock --check` separately verifies that the lock still matches `pyproject.toml`. The lock contains the runtime packages and the optional `dev` extra used by pytest and Ruff; Docker installs only the runtime subset.

When using another PostgreSQL instance, set `DATABASE_URL` in `.env`. The database must already exist. Alembic owns the reviewable schema history. API startup verifies the recorded migration head and never creates, upgrades, or stamps the schema.

Persistence verification

```
$before = Invoke-RestMethod http://localhost:8000/incidents
docker compose restart api
Invoke-RestMethod http://localhost:8000/health
$after = Invoke-RestMethod http://localhost:8000/incidents
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Incident IDs, statuses, logs, and saved RCA reports should remain available. Running detection again without new logs should produce zero new assignments.

Updating dependencies and images

Make dependency updates deliberately from a clean working tree:

```
uv --version
uv lock --upgrade --python 3.13
uv lock --check
uv sync --frozen --extra dev --python 3.13
uv run --frozen pytest -q
uv run --frozen ruff check .
uv run --frozen ruff format --check .
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Review both `pyproject.toml` and `uv.lock`, including platform markers and unexpected transitive changes. To update a container base, inspect the registry manifest, place the verified tag and multi-platform digest in the Dockerfile or Compose file, then rebuild and run the complete checks:

```
docker buildx imagetools inspect python:3.13.16-slim
docker buildx imagetools inspect ghcr.io/astral-sh/uv:0.13.1
docker buildx imagetools inspect postgres:17.11
docker compose config --quiet
docker compose build --no-cache api
docker compose up -d --wait
Invoke-RestMethod http://localhost:8000/health
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Do not merge a lock or digest update until the PostgreSQL-backed test suite, Ruff checks, clean image build, and container health verification all pass. Image updates must not remove the existing `postgres_data` volume.

Startup order and an existing database

The local development command block above assumes the database has already been initialized or baselined. Before launching Uvicorn on a fresh empty database, run the explicit upgrade command from Database Migrations. If tables and records already exist without migration history, follow the reviewed adoption procedure instead.

Do not use the fresh-database upgrade command as a way to baseline existing tables. Do not use stamping to silence a mismatch. Both choices can make the deployed state diverge from the recorded migration history.

What restart and health settings do

The Compose dependency waits for PostgreSQL health before starting the API. The API then verifies its migration revision. Once running, its health probe uses Python urllib against the container's own loopback address. HTTP 503 and connection errors fail that probe.

An unhealthy status alone does not restart a Docker container. The restart policy reacts to container/process lifecycle events. A process can remain running but unhealthy until the dependency recovers or an operator takes action. The configured unless-stopped policy preserves intentional manual stops.

Practical first-run sequence

1. Choose Docker or local Python and install the pinned tools where needed.
2. Create local configuration without committing secrets; keep RCA_PROVIDER disabled for provider-free exploration.
3. Start PostgreSQL and establish migration state using the appropriate fresh or existing-database procedure.
4. Start the API and verify HTTP 200 at /health.
5. Ingest the synthetic log sample and explicitly run detection.
6. Inspect incident evidence and lifecycle endpoints.
7. Use fixture-backed tests to exercise RCA without external calls. A real analyze request with usable evidence requires an enabled configured provider.

07 Environment Variables

Copy `.env.example` to `.env` and edit it locally. Never commit `.env` or API keys.

Variable	Default	Purpose
`DATABASE_URL`	Local TraceMind PostgreSQL URL	SQLAlchemy connection URL
`CORRELATION_WINDOW_SECONDS`	`300`	Maximum fixed span for one incident
`ERROR_BURST_THRESHOLD`	`5`	Error count that adds a burst signal
`SERVICE_DEPENDENCIES`	Payment/user to database	JSON map from downstream service to upstream dependencies
`RCA_PROVIDER`	`disabled`	`disabled`, `gemini`, or `openai`
`GEMINI_API_KEY`	Empty	Gemini credential used only when Gemini is selected
`GEMINI_MODEL`	`gemini-3.5-flash-lite`	Gemini model ID
`OPENAI_API_KEY`	Empty	OpenAI credential used only when OpenAI is selected
`OPENAI_MODEL`	Empty	Structured-output-capable OpenAI model
`RCA_TIMEOUT_SECONDS`	`30`	Provider network timeout
`RCA_MAX_LOGS`	`40`	Maximum selected logs per analysis
`RCA_MAX_EVIDENCE_CHARS`	`40000`	Maximum compact evidence JSON size
`RCA_MAX_OUTPUT_TOKENS`	`4000`	Provider output-token limit

Docker overrides `DATABASE_URL` inside the API container so it can reach the `db` service. Rebuild or recreate the API after changing provider settings:

```
docker compose up -d --build api
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The Dockerfile pins Python 3.13.16 and the uv 0.13.1 installer image by immutable multi-platform digest. Compose pins PostgreSQL 17.11 by digest. A tag keeps the selected version readable; the digest ensures a build or pull retrieves the reviewed image content.

How settings affect investigation

CORRELATION_WINDOW_SECONDS bounds incident membership, while ERROR_BURST_THRESHOLD adds a signal when errors accumulate. SERVICE_DEPENDENCIES describes downstream-to-upstream relationships used for weak linking. These settings influence deterministic evidence grouping before a provider is involved.

RCA_MAX_LOGS and RCA_MAX_EVIDENCE_CHARS bound the provider evidence package. Lower limits reduce the supplied context and may introduce explicit omission gaps and low confidence. Increasing the limits can include more evidence but does not guarantee a better hypothesis.

RCA_TIMEOUT_SECONDS bounds the provider request. RCA_MAX_OUTPUT_TOKENS bounds generated output. A timeout or truncated response is handled as a failure rather than silently retrying another model or provider.

Credentials and provider activation

Keep credentials in local configuration or a deployment secret mechanism. The examples use placeholders. Only the selected provider's credentials are required; validation is lazy at the analysis boundary rather than a credential probe during startup. Provider-free ingestion and correlation remain available with RCA disabled.

08 API Endpoints

Method	Endpoint	Purpose
GET	`/health`	Check API and database readiness
POST	`/logs`	Validate, normalize, and store a log batch
GET	`/logs`	Filter and paginate stored logs
POST	`/incidents/detect`	Run deterministic detection over unassigned logs
GET	`/incidents`	Filter and paginate incidents
GET	`/incidents/{id}`	Retrieve an incident timeline, evidence, and relationships
PATCH	`/incidents/{id}`	Change incident workflow status
POST	`/incidents/{id}/analyze`	Generate, validate, and save an RCA report
GET	`/incidents/{id}/analyses`	Retrieve paginated report history, newest first
GET	`/incidents/{id}/analyses/{report_id}`	Retrieve one report and its evidence snapshot

Common error behavior:

Status	Meaning
404	Requested incident or report does not exist
422	Request validation failed
502	Provider/configuration error, refused or malformed output, or failed RCA validation
503	Database unavailable, RCA disabled, missing provider configuration, or provider quota/rate limit
504	Provider request timed out

Interactive OpenAPI documentation is available at http://localhost:8000/docs while the API is running.

Common operator workflow

Step	Request	What to inspect
1	POST /logs	HTTP 201 and saved IDs. A rejected batch stores none of its entries.
2	POST /incidents/detect	Counts of incidents, assignments and weak relationships.
3	GET /incidents	Find the incident using service/status filters and pagination.
4	GET /incidents/{id}	Timeline, detection evidence and relationships.
5	POST /incidents/{id}/analyze	Accepted report and validation notes, if provider analysis is enabled.
6	GET /incidents/{id}/analyses	Newest-first report history without a provider call.
7	GET /incidents/{id}/analyses/{report_id}	The exact persisted report and its evidence snapshot.

IDs returned by the API should be carried forward between steps; do not assume an incident or report will be assigned ID 1. Repeating ingestion creates new observations, and repeating successful analysis creates new reports. Detection, by contrast, is idempotent for already assigned stored log IDs.

Retry decisions

Correct a 422 input before retrying. For a 502, examine the sanitized failure category and provider configuration rather than treating malformed output as acceptable. A 503 can mean unavailable infrastructure or disabled/missing provider configuration; inspect the response detail. A 504 indicates the provider did not complete within the configured timeout.

The API does not implement request idempotency keys. If a client loses its connection after submitting a write, inspect available history before retrying blindly: the server may already have committed the result.

09 Database Migrations

The initial revision is `20261019_01`. On a new empty database, apply it before starting the API:

```
uv run --frozen python -m app.migrations upgrade
uv run --frozen alembic current
uv run --frozen alembic check
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

`alembic check` must report no new upgrade operations after every model or migration change. The Docker image includes `alembic.ini` and the migration scripts, so the same commands can be used by a dedicated deployment migration job. For a new Docker database, run the migration command once before starting or restarting API replicas:

```
docker compose run --rm api python -m app.migrations upgrade
docker compose up -d --wait
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The TraceMind migration command takes a PostgreSQL session advisory lock before upgrading and releases it after success or failure. PostgreSQL also releases the lock if the migration connection terminates. This serializes concurrent command invocations without holding a lock during normal API operation. API startup compares the database revision with the head loaded from the packaged Alembic scripts. An unbaselined, behind, incompatible, or unreachable database blocks startup with a sanitized error; the API never stamps or upgrades it automatically.

An existing unversioned database must not run `alembic upgrade head`: the initial revision would try to create tables that already exist. Adopt migrations during a maintenance window instead:

1. Back up the database and stop all application writes, API replicas, and migration runners.
2. Compare its tables, columns, constraints, and indexes with the initial revision and current model metadata. Resolve every difference explicitly.
3. When the schemas match, run `alembic stamp 20261019_01` once. Stamping creates the version marker without replaying the initial schema operations; it does not create, alter, or repair tables.
4. Confirm `alembic current` reports `20261019_01 (head)` and `alembic check` reports no changes before restarting application writes.

Never use `stamp` merely to bypass a mismatch. It is safe only after the existing schema has been reviewed and confirmed equivalent to the revision being recorded.

Migration execution, step by step

The maintenance command uses the existing SQLAlchemy engine and Alembic configuration. It obtains a PostgreSQL session advisory lock, commits the lock-acquisition transaction, and supplies that same connection to Alembic. The session lock stays held while Alembic applies pending revisions.

After upgrade, cleanup rolls back any remaining transaction, releases the advisory lock, and commits the unlock operation. Failures are converted to a fixed migration error message. PostgreSQL releases session advisory locks when their owning connection terminates.

Only callers using this maintenance command cooperate with its lock. A direct alembic upgrade or stamp invocation bypasses the wrapper. Existing-database stamping is therefore documented as a single controlled maintenance action with all other migration runners stopped.

Startup state matrix

Recorded database state	Startup result
Exact match with packaged migration heads	Starts normally.
No version table or no recorded heads	Rejects the unbaselined database with baseline guidance.
Older, unknown or otherwise different revision	Rejects the revision mismatch.
Database connection/query failure	Rejects startup with a sanitized verification error.

The expected revision is read from Alembic scripts, not copied into a second application constant. Ordinary API operation does not hold the migration advisory lock.

Safe adoption checklist

Prepare a restorable backup and verify its intended scope before the maintenance window. Stop API replicas and all other writers, verify the schema including constraints and indexes, and confirm that the reviewed revision is the intended baseline. Schema matching should include manual review: automated metadata comparison is not a universal proof of every database property.

After stamping, inspect the recorded revision and schema comparison before resuming writes. Stamping changes migration metadata only. Retain the backup and the review record until post-adoption checks are complete. This document does not authorize stamping or upgrading the existing development database.

10 Testing

The suite uses mocked provider transports; automated tests never make live Gemini or OpenAI calls.

Fast local suite

```
uv lock --check
uv run --frozen pytest -q
uv run --frozen ruff check .
uv run --frozen ruff format --check .
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Without `TEST_DATABASE_URL`, tests use isolated in-memory SQLite databases.

PostgreSQL suite

Start the database and run tests against randomly named schemas. Existing application data is not changed.

```
docker compose up -d db
$env:TEST_DATABASE_URL =
    'postgresql+psycopg://tracemind:tracemind@127.0.0.1:5432/tracemind?connect_timeout=5'
uv run --frozen pytest -q
Remove-Item Env:\TEST_DATABASE_URL
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The database user must be allowed to create and drop schemas.

Focused RCA and provider suite

```
uv run --frozen pytest tests/test_llm.py tests/test_rca.py -q
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Deterministic RCA benchmark

Run the fixture-backed RCA contract benchmark without provider credentials or network access:

```
uv run --frozen pytest -m rca_benchmark -q
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

The benchmark reuses the main RCA regression tests for a valid hypothesis, the local no-evidence result, invalid citations, missing observed facts, duplicate causes, increasing confidence by rank, contradictory primary evidence, provider timeout, omission disclosure, and rejection without persistence. It checks deterministic contract behavior rather than subjective causal accuracy.

Run the same static checks used for RCA changes:

```
uv run --frozen ruff check app/rca.py app/rca_schemas.py app/llm.py tests/test_rca.py tests/test_llm.py
uv run --frozen ruff format --check app/rca.py app/rca_schemas.py app/llm.py tests/test_rca.py
    tests/test_llm.py
git diff --check
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Coverage includes validation and normalization, filtering, atomic batches, deterministic correlation, identifier conflicts, fixed windows, weak relationships, idempotency, lifecycle changes, pagination, transaction rollback, evidence bounds, immutable report history, Gemini and OpenAI wire contracts, malformed output, citation integrity, exact

cause matching, quota and authentication failures, timeouts, response limits, secret sanitization, and PostgreSQL persistence.

The fixture-backed benchmark makes no live Gemini or OpenAI requests. It verifies deterministic contract and persistence behavior; it does not measure causal accuracy, throughput, or latency.

What the suite proves

The shared client fixture uses a random schema when TEST_DATABASE_URL is set. It creates model tables in that test schema and removes the test schema afterward. It deliberately skips production lifespan startup. Dedicated startup and migration tests cover the production revision guard separately.

Without TEST_DATABASE_URL, the ordinary API tests use isolated SQLite memory databases. Migration tests also use temporary SQLite files. The PostgreSQL-specific persistence coverage is skipped when its required URL is unavailable.

The benchmark checks whether deterministic contracts are accepted, normalized or rejected as expected. It does not estimate whether a hypothesis is the true root cause. Live model evaluation, request throughput, latency characterization and host-reboot recovery are separate verification tasks.

Recorded verification context

The preceding audit in this project session reported 208 passing PostgreSQL-backed tests, successful frozen synchronization, lock validation, Ruff, Compose validation, and diff checks. The existing Starlette/httpx deprecation warning remained. Those are historical audit results, not a new test run performed while producing this PDF.

The audit also reported an unbaselined development database containing 14 logs and four RCA reports, with zero ORM metadata differences and the named volume intact. Environment state can change; perform read-only checks before relying on that observation during a deployment.

11 Project Structure

```
app/
  config.py          Environment-backed settings
  correlation.py     Detection and correlation rules
  database.py        SQLAlchemy engine, session, and metadata
  incidents.py       Incident API routes
  llm.py             Gemini and OpenAI provider adapters
  main.py            FastAPI application, startup checks, health, and log routes
  migrations.py      Explicit migration command and revision verification
  models.py          Persistent database models
  rca.py             Evidence construction, validation, and report persistence
  rca_routes.py      RCA API routes
  rca_schemas.py     Strict RCA contracts
  schemas.py         Log and incident contracts
migrations/
  env.py             Alembic runtime configuration
  versions/          Ordered database revisions
data/
  sample_logs.json   Synthetic database/payment failure scenario
docs/
  gemini-investigation.md  Live Gemini request-contract investigation
tests/
  conftest.py
  test_incidents.py
  test_llm.py
  test_logs.py
  test_migrations.py
  test_rca.py
  test_startup.py
alembic.ini          Alembic configuration
compose.yaml         Local API and PostgreSQL services
Dockerfile           Reproducible non-root API image
pyproject.toml       Project metadata and direct dependencies
uv.lock              Cross-platform resolved dependency lock
```

Code is line-wrapped for print. Use the repository source for exact copy/paste commands.

Where to make a change

Start with schemas.py for log/incident contracts, correlation.py for grouping rules, rca_schemas.py and rca.py for analysis acceptance, and llm.py for provider transport behavior. Database model changes belong in models.py with an explicit migration revision and tests; startup should continue to verify rather than mutate the schema.

Keep provider mocks and deterministic regression cases in the existing test modules. A test that validates report rejection should also verify no report row was persisted. Preserve the original prompt version and evidence snapshot on stored reports rather than rewriting historical analysis after a code change.

12 Future Scope

Current limitations

- Detection scans stored logs and memberships in memory and is intended for a development-scale dataset.
- RCA requests are synchronous.
- Provider calls run inline with the request; there is no queue, background execution, or cancellation API.
- Incident detail returns the complete evidence timeline rather than a paginated timeline.
- Burst detection uses a fixed threshold, not a learned baseline.
- Identifier equality and temporal order are correlation evidence, not causal proof.
- There is no authentication, authorization, tenant isolation, rate limiting, or audit identity.
- API startup verifies the Alembic head but does not apply migrations. Existing unversioned databases require the documented one-time schema review and baseline before the updated API can start.
- There is no semantic search over previous incidents or verified resolution knowledge base.
- There is no dashboard or background worker.
- The deterministic RCA benchmark uses mocked providers and fixed fixtures. It does not evaluate live-provider behavior, causal accuracy, throughput, or latency.
- The PostgreSQL persistence test is skipped when `TEST_DATABASE_URL` is unavailable; the remaining tests use isolated in-memory SQLite databases.
- TraceMind does not confirm causes or execute remediation.

PostgreSQL advisory transaction locking serializes detector runs so concurrent workers cannot assign the same stored log twice.

Potential future work includes semantic search over prior incidents, a reviewed resolution knowledge base, asynchronous RCA jobs, authentication and tenant isolation, operational dashboards, scalable detection for larger datasets, and separately approved live-provider quality evaluation. Automated remediation should be considered only with explicit review and operational safeguards.

Suggested sequencing for future work

First establish safe adoption of the migration baseline in the target environment. Then address access control and operational ownership before exposing the API beyond its local-development boundary. Scaling, asynchronous jobs, richer retrieval and a user interface can each be evaluated as separate changes with concrete acceptance criteria.

For model evaluation, build a reviewed dataset with known investigation outcomes, define what evidence the model is allowed to see, and separate factual grounding from causal accuracy. Such evaluation requires explicit approval for external provider calls and cannot be inferred from passing mock-provider tests.

13 License

The inspected repository does not include a license file. Consult the repository owner for licensing terms.

Repository status

No license file was found in the inspected repository. This handbook does not add or select a software license. Licensing terms should be specified by the repository owner in the source repository before a distribution policy is assumed.

14 Worked Investigation

Synthetic database/payment incident

The repository's data/sample_logs.json contains four synthetic events. The first three form a database/payment failure sequence sharing trace-payment-001; the fourth is an unrelated warning. This example illustrates the implemented rules without claiming a verified real-world cause.

Time (UTC)	Service	Observation
10:00:01	database-service	Connection pool exhausted; PoolExhaustedError.
10:00:02	payment-service	Database request timed out; same trace, request req-payment-001.
10:00:03	payment-service	Payment processing failed; same request and trace.
10:00:04	user-service	Latency warning with request req-user-002 and no shared trace.

What detection can establish

The shared trace and timestamps support deterministic grouping of the first three error records. Service names and severity values are normalized during ingestion. The warning remains stored and queryable, but it does not seed this incident. The detector can expose common trace evidence, multiple services and repeated observations; none of these alone proves which event initiated the incident.

What a useful hypothesis would contain

An analyst might investigate whether pool exhaustion contributed to the payment timeouts. An accepted report would need observed facts tied to supplied log IDs, a first-ranked cause whose text exactly matches probable_cause, and identical ordered citations at both levels. It should distinguish the exhaustion observation from an unverified explanation for why exhaustion occurred.

Recommended checks could investigate pool configuration, connection release behavior, database saturation and request concurrency. These checks are illustrative investigation suggestions, not operations executed by TraceMind and not new implemented product features.

How to challenge the report

1. Read each cited event in the stored snapshot.
2. Check whether timestamps and identifiers support the proposed relationship.
3. Look for competing explanations and contradictory evidence.
4. Review omission gaps before concluding the snapshot is representative.
5. Compare reports over time using history rather than overwriting an earlier report.
6. Record the operational status separately from any decision about causal certainty.

Why the warning is still useful

Keeping unrelated observations queryable preserves the ability to investigate other incidents later. Avoiding forced grouping reduces unsupported conclusions. If subsequent evidence establishes a relationship, that is new investigation context; it does not retroactively rewrite the evidence snapshot used by an existing report.

15 Troubleshooting & Source Notes

Troubleshooting map

Symptom	Read-only first check	Appropriate next step
API rejects startup as unbaselined	Inspect migration history and compare schema to the initial revision.	Choose reviewed baseline adoption for existing tables; upgrades for an empty database.
API rejects a revision mismatch	Compare recorded revision with packaged Alembic heads.	Run the explicit maintenance upgrade only against the intended approved target.
Running API returns /health 503	Inspect container health and sanitized API/database logs.	Restore connectivity; do not reset persistent storage as a diagnostic shortcut.
Analysis returns 503 while logs work	Inspect RCA_PROVIDER and selected provider configuration.	Enable the intended provider only when authorized; keep credentials private.
Analysis returns 502	Inspect structured failure category and relevant contract test.	Fix configuration or rejected output behavior without weakening validation.
Provider call times out	Review timeout and dependency availability.	Decide explicitly whether to retry; the adapter has no automatic retry/fallback.
A new report is low confidence	Read validation_notes and evidence_gaps.	Investigate sparse/truncated or contradictory evidence before accepting a cause.
Tests skip PostgreSQL coverage	Inspect TEST_DATABASE_URL availability.	Use the documented PostgreSQL test workflow with isolated schemas.

Source map

Repository source	Used for
README.md	Product overview, commands, endpoint list, environment table and limits.
app/rca.py and app/rca_schemas.py	Citation/order checks, confidence normalization, observed facts, gaps and output bounds.
app/models.py	Physical table names, integer keys, relationships and immutable report fields.
app/migrations.py and migrations/env.py	Explicit upgrade, shared connection, lock cleanup and expected-head verification.
app/main.py	Lifespan startup guard and health behavior.
tests/conftest.py and migration/startup tests	Isolated test schemas, skipped production lifespan and targeted verification.
data/sample_logs.json	Synthetic investigation walkthrough.
pyproject.toml, uv.lock, Dockerfile and compose.yaml	Locked dependencies, packaged runtime and local service assumptions.

Documentation boundaries

This is a source-based technical guide, not a fresh deployment certification. No database schema, application record, container or volume was changed to generate it. No live Gemini/OpenAI call was made. The commands are examples for an operator to execute against an explicitly chosen environment.

The diagrams are rendered summaries of the README diagrams. Code blocks are line-wrapped for page layout; consult the repository for exact runnable formatting. Future-scope items are proposals and should not be read as implemented behavior.

Local installation, database adoption and provider activation are different responsibilities. Keep their approvals and verification evidence explicit when moving from a developer workstation to a shared deployment.